- Temporal LSTM for Dynamic Ap/Tp Prediction
  - Technical Documentation with Mathematical Foundations
  - 1. Model Overview
    - 1.1 Problem Formulation
    - 1.2 Multi-Task Learning Objective
  - 2. Architecture Specification
    - 2.1 Input Tensors
  - 3. Layer-by-Layer Technical Analysis
    - 3.1 Input Layer: Event Sequence Encoding
      - Mathematical Representation
      - Padding Strategy
    - 3.2 Input Layer: Global Features
      - Era Definitions
    - 3.3 Input Layer: Persona Features
    - 3.4 Persona Embedding Layer
      - Architecture
      - ReLU Activation
      - Why Embedding?
    - 3.5 LSTM Layers: Sequence Processing
      - LSTM Cell Mathematics
      - Architecture Configuration
      - Output Extraction
      - Why 2 Layers?
    - 3.6 Feature Concatenation
    - 3.7 Shared Fully Connected Layers
      - FC1: Expansion Layer
      - FC2: Compression Layer
      - Dropout Regularization
      - Why This Architecture?
    - 3.8 Multi-Task Prediction Heads
      - Churn Head
      - Ap Head (Activity Threshold)
      - Tp Head (Time Window)
  - 4. Loss Functions: Mathematical Details
    - 4.1 Binary Cross-Entropy (Churn)
    - 4.2 Mean Squared Error (Ap, Tp)
    - 4.3 Combined Loss with Weights

- 5. Training Procedure
  - 5.1 Optimization
  - 5.2 Learning Rate Schedule
  - 5.3 Gradient Clipping
  - 5.4 Early Stopping
  - 5.5 Data Sampling Strategy
- 6. Model Capacity Analysis
  - 6.1 Parameter Count
  - 6.2 Computational Complexity
- 7. Inference Pipeline
  - 7.1 Single User Prediction
  - 7.2 Batch Inference
- 8. Churn Label Definition
  - 8.1 Temporal Labeling Scheme
- 9. Era-Aware Temporal Features
  - 9.1 Era Encoding Rationale
  - 9.2 Why Era Matters for Churn
- 10. Persona-Specific Thresholds
  - 10.1 Why Personalization?
  - 10.2 Multi-Task Benefit
- 11. Architectural Design Decisions
  - 11.1 Why LSTM over Transformer?
  - 11.2 Why Multi-Task over Separate Models?
  - 11.3 Why Persona Embeddings over One-Hot?
- 12. Summary
  - 12.1 Complete Forward Pass
  - 12.2 Key Innovations
- 13. Files Reference

Temporal LSTM for Dynamic Ap/Tp Prediction

Technical Documentation with Mathematical Foundations

---

1. Model Overview

1.1 Problem Formulation

Given a user u with persona $p \in \{0, 1, ..., 19\}$ and an observation cutoff time t_c, we aim to predict:

1. **Churn Probability**: $P(\text{churn}|u, t_c) \in [0, 1]$
2. **Activity Threshold**: $A_p \in [0, 1]$ — minimum activity ratio indicating engagement
3. **Time Window**: $T_p \in \mathrm{R}^+$ — days of inactivity before declaring churn

1.2 Multi-Task Learning Objective

The model minimizes a weighted combination of task-specific losses:

$$\mathrm{L}_{total} = \lambda_1 \cdot \mathrm{L}_{churn} + \lambda_2 \cdot \mathrm{L}_{Ap} + \lambda_3 \cdot \mathrm{L}_{Tp}$$

Where:

- $\lambda_1 = 2.0$ (Binary Cross-Entropy)
- $\lambda_2 = 1.0$ (Mean Squared Error)
- $\lambda_3 = 0.1$ (Mean Squared Error, scaled)

---

2. Architecture Specification

2.1 Input Tensors

Input	Shape	Description
Event Sequence	$(B, 50, 2)$	Batch of sequences: gain values + time deltas
Global Features	$(B, 4)$	Temporal context features
Persona Features	$(B, 10)$	Statistical features per persona

Where B is the batch size (256 during training).

---

3. Layer-by-Layer Technical Analysis

3.1 Input Layer: Event Sequence Encoding

Mathematical Representation

For user u, we extract the last $L = 50$ events before cutoff t_c:

$$\mathbf{X}^{(u)} = \begin{bmatrix} g_1 & \Delta t_1 \\ g_2 & \Delta t_2 \\ \vdots & \vdots \\ g_{50} & \Delta t_{50} \end{bmatrix} \in \mathrm{R}^{50\times 2}$$

Where:

- g_i = gain/reputation value of event i
- $\Delta t_i = (t_i - t_{i-1})/86400$ = days between consecutive events

Padding Strategy

For users with fewer than 50 events, we apply **left-padding**:

$$\mathbf{X}^{(u)}_{padded} = [\underbrace{0, ..., 0}_{50-n \text{ zeros}}, x_1, x_2, ..., x_n]$$

> **Stack Overflow Analogy**: Think of this as a user's "activity timeline" — each row is one contribution (answer, question, comment, edit). The gain shows how valuable it was (upvotes received), and time_delta shows the gap since their previous action. A user who answers daily will have small Δt values; someone answering once a month will have $\Delta t \approx 30$.

3.2 Input Layer: Global Features

The global feature vector captures temporal context:

$$\mathbf{g}^{(u)} = \begin{bmatrix} d_{recency} \\ e_{era} \\ m_{norm} \\ y_{norm} \end{bmatrix} \in \mathrm{R}^4$$

Feature	Formula	Range
Days since last event	$d_{recency} = (t_c - t_{last})/86400$	$[0, \infty)$
Era encoding	$e_{era} \in \{0 : \text{pre-COVID}, 1 : \text{COVID}, 2 : \text{post-LLM}\}$	$\{0, 1, 2\}$
Normalized month	$m_{norm} = \text{month}(t_c)/12$	$[0.083, 1.0]$
Normalized year	$y_{norm} = (\text{year}(t_c) - 2016)/7$	$[0, 1]$

Era Definitions

Era	Years	Rationale
Pre-COVID	2016-2019	Baseline community behavior
COVID	2020-2021	Remote work → increased online activity
Post-LLM	2022-2024	ChatGPT impact → changed Q&A dynamics

> **Stack Overflow Analogy**: The "era" matters because SO member behavior shifted dramatically. During COVID, developers worked from home and asked more questions. After ChatGPT launched, many simple questions moved to AI — changing what "normal activity" looks like. A user who was active in 2019 but quiet in 2023 might not be churning; they might just be using AI for quick questions now.

3.3 Input Layer: Persona Features

Each of the 20 personas has a statistical profile derived from NMF clustering:

$$\mathbf{p}^{(k)} = \begin{bmatrix} \log(1+\bar{r}_k) \\ \log(1+\bar{u}_k) \\ \log(1+\bar{a}_k) \\ \log(1+\bar{q}_k) \\ \bar{d}_k/190 \\ \rho_{coll,k} \\ \rho_{indiv,k} \\ 1[H_k > 0] \\ 1[C_k > 0] \\ 1[T_k > 0] \end{bmatrix} \in \mathrm{R}^{10}$$

Where:

- $\bar{r}_k$ = average reputation for persona k
- $\bar{u}_k$ = average upvotes received
- $\bar{a}_k$ = average answer count
- $\bar{q}_k$ = average question count
- $\bar{d}_k$ = average tenure duration (normalized by max 190 months)
- $\rho_{coll,k}$ = collectivistic ratio (helping others)
- $\rho_{indiv,k}$ = individualistic ratio (asking for self)
- H, C, T = binary flags for Helper/Curator/Teacher archetypes

> **Stack Overflow Analogy**: Personas capture "types" of SO members. Persona 0 might be "Jon Skeet types" — high reputation, thousands of answers, rarely asks questions. Persona 15 might be "Drive-by askers" — low rep, many questions, few answers. Each type has different churn patterns: a prolific answerer going quiet for 30 days is concerning; a casual asker being quiet for 30 days is normal.

3.4 Persona Embedding Layer

Architecture

$$\mathbf{e}^{(u)} = \mathrm{ReLU}(\mathbf{W}_e \cdot \mathbf{p}^{(k)} + \mathbf{b}_e)$$

Where:

- $\mathbf{W}_e \in \mathrm{R}^{32\times 10}$ — learnable weight matrix
- $\mathbf{b}_e \in \mathrm{R}^{32}$ — learnable bias vector
- Output: $\mathbf{e}^{(u)} \in \mathrm{R}^{32}$

ReLU Activation

$$\text{ReLU}(x) = \max(0, x)$$

This introduces non-linearity and ensures non-negative embeddings.

Why Embedding?

Raw persona statistics are **shallow features**. The embedding layer learns a **latent representation** that captures:

- Non-linear interactions between features
- Task-relevant combinations (e.g., "high rep + low recent activity = different from low rep + low activity")

> **Stack Overflow Analogy**: A raw stat like "500 answers" means different things for different users. For a 10-year member, it's modest (50/year). For a 1-year member, it's prolific. The embedding learns to combine tenure, answer count, and reputation into a meaningful "user engagement profile" — not just raw numbers, but their *implications*.

3.5 LSTM Layers: Sequence Processing

LSTM Cell Mathematics

For each timestep t in the sequence, the LSTM computes:

Forget Gate — What to discard from memory:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

Input Gate — What new information to store:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

Candidate Cell State — Proposed new memory:

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

Cell State Update — Actual memory update:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

Output Gate — What to expose as hidden state:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

Hidden State — Layer output:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Where:

- σ = sigmoid function: $\sigma(x) = 1/(1 + e^{-x})$
- $\odot$ = Hadamard (element-wise) product
- $[\cdot, \cdot]$ = concatenation operation

Architecture Configuration

Parameter	Value
Input dimension	2 (gain, time_delta)
Hidden dimension	128
Number of layers	2 (stacked)
Dropout	0.2 (between layers)
Bidirectional	No

Output Extraction

We use the **final hidden state** from the **last layer**:

$$\mathbf{h}_{seq} = \mathbf{h}_{T=50}^{(L=2)} \in \mathrm{R}^{128}$$

> **Stack Overflow Analogy**:
>
> The LSTM reads through a user's last 50 contributions like scrolling through their profile history:
>
> - **Forget Gate**: "This user had a burst of activity 2 years ago during a specific project — that's less relevant now, let's downweight it."
> - **Input Gate**: "They just earned a gold badge! This is significant new information."
> - **Cell State**: The running "story" of this user's engagement trajectory.
> - **Output Gate**: "What's the key takeaway at this point in their history?"

> After reading all 50 events, $\mathbf{h}_{seq}$ captures the "narrative arc" — are they ramping up? Winding down? Stable contributor? Sporadic visitor?

Why 2 Layers?

Stacked LSTMs learn hierarchical temporal patterns:

- **Layer 1**: Low-level patterns (daily/weekly rhythms)
- **Layer 2**: High-level patterns (seasonal trends, life events, career changes)

3.6 Feature Concatenation

All feature streams are concatenated into a unified representation:

$$\mathbf{z} = [\mathbf{h}_{seq} \| \mathbf{g} \| \mathbf{e}] \in \mathrm{R}^{164}$$

Component	Dimension	Information
$\mathbf{h}_{seq}$	128	Temporal activity patterns
$\mathbf{g}$	4	Temporal context (era, recency)
$\mathbf{e}$	32	Persona embedding
Total	**164**	Complete user representation

> **Stack Overflow Analogy**: This is like combining three reports about a user:
>
> 1. **Activity Timeline Analysis** (LSTM): "User was highly active 2019-2020, then gradually decreased. Last answer was 45 days ago."
> 2. **Temporal Context** (Global): "We're evaluating in 2023, post-LLM era. It's January (New Year slump typical)."
> 3. **User Profile** (Embedding): "This is a 'Weekend Warrior' type — typically answers 2-3 questions on Saturdays, rarely during weekdays."

3.7 Shared Fully Connected Layers

FC1: Expansion Layer

$$\mathbf{a}_1 = \mathrm{Dropout}_{0.3}(\mathrm{ReLU}(\mathbf{W}_1 \cdot \mathbf{z} + \mathbf{b}_1))$$

Where:

- $\mathbf{W}_1 \in \mathrm{R}^{256\times164}$
- $\mathbf{b}_1 \in \mathrm{R}^{256}$
- Output: $\mathbf{a}_1 \in \mathrm{R}^{256}$

FC2: Compression Layer

$$\mathbf{a}_2 = \text{Dropout}_{0.2}(\text{ReLU}(\mathbf{W}_2 \cdot \mathbf{a}_1 + \mathbf{b}_2))$$

Where:

- $\mathbf{W}_2 \in \mathrm{R}^{128\times256}$
- $\mathbf{b}_2 \in \mathrm{R}^{128}$
- Output: $\mathbf{a}_2 \in \mathrm{R}^{128}$

Dropout Regularization

During training, dropout randomly zeros elements with probability p:

$$\text{Dropout}_p(\mathbf{x})_i = \begin{cases} 0 & \text{with probability } p \\ x_i/(1-p) & \text{with probability } 1-p \end{cases}$$

The scaling by $1/(1-p)$ ensures expected values remain consistent between training and inference.

Why This Architecture?

1. **Expansion (164→256)**: Creates "overcomplete" representation for rich feature interactions
2. **Compression (256→128)**: Forces model to learn compact, essential features
3. **Shared layers**: All three tasks benefit from the same learned representation

> **Stack Overflow Analogy**: Think of this as the "analysis department" that processes all the raw intelligence:
>
> - **FC1 (expansion)**: Generates many hypotheses — "Could be burnout," "Could be career change," "Could be switching to another platform," "Could be seasonal."
> - **FC2 (compression)**: Distills to key factors — "Most likely: gradual disengagement pattern consistent with finding alternative resources (LLM

> tools)."
>
> **Dropout** prevents overfitting to specific users. Without it, the model might memorize: "User #12345 always churns in January" instead of learning general patterns.

3.8 Multi-Task Prediction Heads

Each head is a small MLP (Multi-Layer Perceptron) specialized for one output:

Churn Head

$$\hat{y}_{churn} = \sigma(\mathbf{W}_{c2} \cdot \text{ReLU}(\mathbf{W}_{c1} \cdot \mathbf{a}_2 + \mathbf{b}_{c1}) + \mathbf{b}_{c2})$$

Layer	Dimensions	Activation
Linear	128 → 64	ReLU
Linear	64 → 1	Sigmoid

Output: $\hat{y}_{churn} \in (0, 1)$ — probability of churn

Ap Head (Activity Threshold)

$$\hat{y}_{Ap} = \sigma(\mathbf{W}_{a2} \cdot \text{ReLU}(\mathbf{W}_{a1} \cdot \mathbf{a}_2 + \mathbf{b}_{a1}) + \mathbf{b}_{a2})$$

Layer	Dimensions	Activation
Linear	128 → 64	ReLU
Linear	64 → 1	Sigmoid

Output: $\hat{y}_{Ap} \in (0, 1)$ — activity threshold ratio

Tp Head (Time Window)

$$\hat{y}_{Tp} = \text{ReLU}(\mathbf{W}_{t2} \cdot \text{ReLU}(\mathbf{W}_{t1} \cdot \mathbf{a}_2 + \mathbf{b}_{t1}) + \mathbf{b}_{t2})$$

Layer	Dimensions	Activation
Linear	128 → 64	ReLU
Linear	64 → 1	ReLU

Output: $\hat{y}_{Tp} \in [0, \infty)$ — time window in days

> **Stack Overflow Analogy**: Three specialized "expert panels" review the same evidence:
>
> - **Churn Panel**: "Based on declining activity, post-LLM era, and 'Casual Answerer' profile, we assess 73% churn probability."
> - **Ap Panel**: "For this user type, historically active 2x/week, we recommend Ap=0.35 — if they drop below 35% of their normal rate, flag them."
> - **Tp Panel**: "Given their sporadic pattern (sometimes 3-week gaps normally), set Tp=45 days before declaring churn."

---

4. Loss Functions: Mathematical Details

4.1 Binary Cross-Entropy (Churn)

$$L_{churn} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Where:

- $y_i \in \{0, 1\}$ = true label (churned or not)
- $\hat{y}_i \in (0, 1)$ = predicted probability

Properties:

- Heavily penalizes confident wrong predictions
- Suitable for imbalanced classes (churn rate ~30-40%)

4.2 Mean Squared Error (Ap, Tp)

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

For Tp, we apply scaling to prevent gradient explosion:

$$\mathrm{L}_{Tp} = \frac{1}{N} \sum_{i=1}^{N} \left(\frac{y_i}{100} - \frac{\hat{y}_i}{100} \right)^2$$

4.3 Combined Loss with Weights

$$\mathrm{L}_{total} = 2.0 \cdot \mathrm{L}_{churn} + 1.0 \cdot \mathrm{L}_{Ap} + 0.1 \cdot \mathrm{L}_{Tp}$$

Weight Rationale:

Task	Weight	Justification
Churn	2.0	Primary objective; highest business impact
Ap	1.0	Important for personalization; bounded output
Tp	0.1	Secondary; large numerical range needs dampening

---

5. Training Procedure

5.1 Optimization

Optimizer: Adam (Adaptive Moment Estimation)

$$\begin{aligned}
\mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla_\theta \mathrm{L} \\
\mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla_\theta \mathrm{L})^2 \\
\hat{\mathbf{m}}_t &= \mathbf{m}_t / (1 - \beta_1^t) \\
\hat{\mathbf{v}}_t &= \mathbf{v}_t / (1 - \beta_2^t) \\
\theta_{t+1} &= \theta_t - \alpha \cdot \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon)
\end{aligned}$$

Hyperparameters:

- $\alpha = 0.001$ (learning rate)
- $\beta_1 = 0.9, \beta_2 = 0.999$
- $\epsilon = 10^{-8}$

5.2 Learning Rate Schedule

ReduceLROnPlateau:

- Monitor: Validation loss
- Factor: 0.5 (halve LR when plateau detected)
- Patience: 5 epochs

$$\alpha_{new} = \begin{cases} 0.5 \cdot \alpha & \text{if no improvement for 5 epochs} \\ \alpha & \text{otherwise} \end{cases}$$

5.3 Gradient Clipping

To prevent exploding gradients in LSTMs:

$$\nabla_\theta \mathrm{L} \leftarrow \nabla_\theta \mathrm{L} \cdot \min\left(1, \frac{1.0}{\|\nabla_\theta \mathrm{L}\|_2}\right)$$

This ensures $\|\nabla_\theta \mathrm{L}\|_2 \leq 1.0$.

5.4 Early Stopping

- **Patience**: 10 epochs
- **Criterion**: Validation loss
- **Checkpoint**: Best model saved at lowest validation loss

5.5 Data Sampling Strategy

Parameter	Value	Purpose
Cutoff years	2017-2022	Multiple evaluation points per user
Max users/persona	500	Memory management, class balance
Min events/user	5	Ensure meaningful sequences
Train/Val split	80/20	Standard validation setup

> **Stack Overflow Analogy**: We train by "replaying history" — for each year from 2017-2022, we pretend it's January 1st and ask: "Given what we knew about this user up to now, would they churn in the following year?" This gives us multiple training examples per user and teaches the model to handle different eras.

6. Model Capacity Analysis

6.1 Parameter Count

Component	Parameters	Calculation
Persona Embedding	352	$10 \times 32 + 32$
LSTM Layer 1	67,072	$4 \times [(2 + 128) \times 128 + 128]$
LSTM Layer 2	131,584	$4 \times [(128 + 128) \times 128 + 128]$
FC1	42,240	$164 \times 256 + 256$
FC2	32,896	$256 \times 128 + 128$
Churn Head	8,257	$(128 \times 64 + 64) + (64 \times 1 + 1)$
Ap Head	8,257	Same as Churn
Tp Head	8,257	Same as Churn
Total	**~165,000**	

6.2 Computational Complexity

Forward Pass: $O(L \cdot H^2)$ for LSTM, where $L = 50$, $H = 128$

Memory: $O(B \cdot L \cdot H)$ for storing hidden states during backprop

---

7. Inference Pipeline

7.1 Single User Prediction

```
def predict(user_sequence, global_features, persona_id):
    """
    Inputs:
        user_sequence: tensor (50, 2) - gain, time_delta
        global_features: tensor (4,) - recency, era, month, year
```

```
        persona_id: int - persona cluster assignment

    Outputs:
        churn_prob: float in (0, 1)
        ap_threshold: float in (0, 1)
        tp_window: float > 0 (days)
    """
    # Load persona features
    persona_feat = persona_features_matrix[persona_id]

    # Forward pass
    churn_prob, ap_pred, tp_pred = model(
        sequence=user_sequence.unsqueeze(0),
        global_feat=global_features.unsqueeze(0),
        persona_feat=persona_feat.unsqueeze(0)
    )

    return churn_prob.item(), ap_pred.item(), tp_pred.item()
```

7.2 Batch Inference

For production, batch users by persona for efficient GPU utilization:

$$\text{Throughput} \approx \frac{B \cdot 1000}{T_{forward}} \text{ users/second}$$

With $B = 256$ and $T_{forward} \approx 10ms$ on GPU: ~25,000 users/second.

---

8. Churn Label Definition

8.1 Temporal Labeling Scheme

For cutoff time t_c:

$$y_{churn} = \begin{cases} 1 & \text{if no events in } [t_c + 90\text{d}, t_c + 455\text{d}] \\ 0 & \text{otherwise} \end{cases}$$

Where:

- **Grace period**: 90 days (natural gaps allowed)
- **Observation window**: 365 days after grace period

> **Stack Overflow Analogy**:
>
> We give users a 90-day "benefit of the doubt" — maybe they're on vacation, parental leave, or between jobs. After that, we look at the next year: if they made zero contributions in 365 days, they've churned.
>
> This is generous because Stack Overflow has many "seasonal" users — students who only contribute during semesters, professionals who contribute during specific projects.

---

9. Era-Aware Temporal Features

9.1 Era Encoding Rationale

Era	Code	Stack Overflow Context
Pre-COVID (2016-2019)	0	Stable growth phase; traditional Q&A dominant
COVID (2020-2021)	1	Remote work surge; 40% traffic increase; new user influx
Post-LLM (2022-2024)	2	ChatGPT impact; simple questions decline; expert content valued more

9.2 Why Era Matters for Churn

The **same behavior** has different implications across eras:

Behavior	Pre-COVID Interpretation	Post-LLM Interpretation
30% activity drop	Likely disengaging	Possibly normal (less simple questions)
No questions asked	Might be expert-only	Might be using AI for questions

Behavior	Pre-COVID Interpretation	Post-LLM Interpretation
Long response times	Slow platform engagement	Competition from AI assistants

The model learns era-specific patterns, allowing it to calibrate predictions appropriately.

---

10. Persona-Specific Thresholds

10.1 Why Personalization?

Different user types have fundamentally different engagement patterns:

Persona Type	Typical Ap	Typical Tp	Rationale
Power Answerer	0.6-0.8	14-21 days	High baseline; quick churn detection
Casual Contributor	0.2-0.4	45-60 days	Sporadic normally; longer window needed
Question Asker	0.3-0.5	30-45 days	Project-driven; moderate thresholds
Curator/Editor	0.4-0.6	21-30 days	Consistent but lower volume

10.2 Multi-Task Benefit

By predicting Ap and Tp alongside churn, the model:

1. **Shares representations**: Same features inform all predictions
2. **Regularizes learning**: Multiple objectives prevent overfitting
3. **Enables personalization**: Per-user thresholds, not global rules

---

11. Architectural Design Decisions

11.1 Why LSTM over Transformer?

Factor	LSTM	Transformer
Sequence length (50)	Efficient	Overkill for short sequences
Training data size	Works with moderate data	Needs more data
Temporal ordering	Naturally sequential	Requires positional encoding
Interpretability	Hidden state = "memory"	Attention weights harder to interpret
Computational cost	Lower	Higher (self-attention is $O(L^2)$)

Conclusion: For sequences of 50 events with ~100K training samples, LSTM is more appropriate.

11.2 Why Multi-Task over Separate Models?

Approach	Pros	Cons
Separate models	Task-specific optimization	No knowledge sharing; 3x parameters
Multi-task	Shared features; regularization; efficiency	Potential task interference

Our weighted loss ($\lambda_1 > \lambda_2 > \lambda_3$) mitigates task interference by prioritizing churn prediction.

11.3 Why Persona Embeddings over One-Hot?

Encoding	Dimensions	Properties
One-hot	20	Sparse; no similarity notion

Encoding	Dimensions	Properties
Statistical features	10	Dense but linear
Learned embedding	32	Dense; captures non-linear interactions

The embedding layer learns that similar personas (e.g., "Prolific Answerer" and "Moderate Answerer") should have similar representations, enabling better generalization.

---

12. Summary

12.1 Complete Forward Pass

$$
\begin{aligned}
\mathbf{e} &= \text{ReLU}(\mathbf{W}_e\mathbf{p} + \mathbf{b}_e) && \text{(Persona embedding)} \\
\mathbf{h}_{50} &= \text{LSTM}(\mathbf{X}) && \text{(Sequence encoding)} \\
\mathbf{z} &= [\mathbf{h}_{50} \| \mathbf{g} \| \mathbf{e}] && \text{(Concatenation)} \\
\mathbf{a}_2 &= \text{FC}_2(\text{FC}_1(\mathbf{z})) && \text{(Shared processing)} \\
\hat{y}_{churn} &= \text{Head}_{churn}(\mathbf{a}_2) && \text{(Churn prediction)} \\
\hat{y}_{Ap} &= \text{Head}_{Ap}(\mathbf{a}_2) && \text{(Ap prediction)} \\
\hat{y}_{Tp} &= \text{Head}_{Tp}(\mathbf{a}_2) && \text{(Tp prediction)}
\end{aligned}
$$

12.2 Key Innovations

1. **Era-aware temporal modeling**: Captures platform evolution impact
2. **Persona-specific thresholds**: Personalized Ap/Tp for each user type
3. **Multi-task learning**: Efficient, regularized joint prediction
4. **Sequence-based churn detection**: Learns activity trajectory patterns

---

13. Files Reference

File	Description
`lstm_temporal_ap_tp.py`	Model implementation and training script

File	Description
`best_temporal_lstm_model.pt`	Trained model weights
`lstm_model_metadata.pkl`	Configuration and preprocessing artifacts
`lstm_architecture_detailed.png`	Architecture visualization
`lstm_architecture_diagram.md`	Mermaid-based diagrams

Technical Documentation v1.0

Model: TemporalChurnPredictor

Framework: PyTorch